COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	9/4/2026	**DATE SUBMITTED:**	9/9/2026
NAME:	Ralph Angelov F. Braganza	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 7.1 Sorting Algorithms Pt1

1. PROCEDURE OUTPUT

ILO A: Create C++ code for implementation of selection, bubble, insertion.

Code + Console Screenshot

Code:

main.cpp

main.cpp

Sorting > main.cpp > main()

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "sorting.h"

const int SIZE = 100;

int main(){

    // Array of size 100 to hold random integers
    int arr[SIZE];
    srand(time(0));

    for (int i = 0; i < SIZE; i++){
        arr[i] = rand()%100;
    }
```

sorting.h

sorting.h

Sorting > sorting.h > selectionSort<T>(T [], int)

```cpp
#ifndef SORTING_H
#define SORTING_H

#include <iostream>

// Function Prototypes

// Function to display the array
void displayArray(int arr[], int size)
{
    for (int i = 0; i < size; i++){
        std::cout << arr[i] << " ";
    }
}
```

Console Screenshot:

```
UNSORTED ARRAY:
67 43 58 49 15 79 94 85 53 99 66 30 74 75 98 38 6 75 91 52 55 6 83 20 17 39 72 24 22 55 25 82 4 67 3
8 51 95 91 16 79 57 54 76 90 45 68 39 2 8 38 62 94 38 5 22 23 26 30 22 24 92 49 72 12 83 99 58 55 76
 0 58 52 45 0 60 6 89 9 61 12 23 10 0 34 62 30 93 45 56 48 29 10 81 34 61 59 57 44 60 49
------------------------------------------------------------
```

Observations

An array of 100 elements is created in the main program (main.cpp), and this array is filled up using random values between 0 and 99 using rand() % 100. Next, the function to display all these elements displayArray(arr, SIZE) is used to

	display all the 100 elements of the array before and after applying any sort method. The function displayArray() in the sorting program (sorting.h) has a for loop in which i = 0 and keeps on increasing until i < size. Thus, all the array elements will be displayed from arr[0] to arr[99].

Table 7-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot	**Code:**
	main.cpp
	`// Displaying Bubble Sort`
	`std::cout << "\nBUBBLE SORT";`
	`std::cout << "\n\nUnsorted Array:\n";`
	`displayArray(arr, SIZE);`
	`bubbleSort(arr, SIZE);`
	`std::cout << "\n\nSorted Array Using Bubble Sort:\n";`
	`displayArray(arr, SIZE);`
	sorting.h
	`// Function to perform Bubble Sort`
	`template <typename T>`
	`void bubbleSort(T arr[], int maxSize){`
	`for(int i = 0; i < maxSize; i++){`
	`for(int j = i + 1; j < maxSize; j++){`
	`if(arr[i]>arr[j]){`
	`std::swap(arr[j],arr[i]);`
	`}`
	`}`
	`}`
	`}`
	Console Screenshot:
	`BUBBLE SORT`
	`Unsorted Array:`
	`67 43 58 49 15 79 94 85 53 99 66 30 74 75 98 38 6 75 91 52 55 6 83 20 17 39 72 24 22 55 25 82 4 67 3 8 51 95 91 16 79 57 54 76 90 45 68 39 2 8 38 62 94 38 5 22 23 26 30 22 24 92 49 72 12 83 99 58 55 76 0 58 52 45 0 60 6 89 9 61 12 23 10 0 34 62 30 93 45 56 48 29 10 81 34 61 59 57 44 60 49`
	`Sorted Array Using Bubble Sort:`
	`0 0 0 2 4 5 6 6 6 8 9 10 10 12 12 15 16 17 20 22 22 22 23 23 24 24 25 26 29 30 30 30 34 34 38 38 38 38 39 39 43 44 45 45 45 48 49 49 49 51 52 52 53 54 55 55 55 56 57 57 58 58 58 59 60 60 61 61 62 62 6 6 67 67 68 72 72 74 75 75 76 76 79 79 81 82 83 83 85 89 90 91 91 92 93 94 94 95 98 99 99`
	`--------------------------------------------------------`

Observations	An array of 100 random numbers is generated in main.cpp. Then, the function displayArray(arr, SIZE) is invoked to display the array of unsorted numbers before implementing the Bubble Sort algorithm. Later, the function bubbleSort(arr, SIZE) is invoked to generate the sorted array. In sorting.h, there are two for loops in the Bubble Sort algorithm such that the first one runs 100 times and the second one compares the number with others in the array. If arr[i] > arr[j], then both numbers will be swapped by each other using std::swap(). The number of comparisons can be approximated to be equal to 100 * 99 / 2 = 4,950, which gives $O(n^2)$ time complexity.

Table 7-2. Bubble Sort Technique

Code + Console Screenshot	**Code:**
	main.cpp
	`main.cpp`
	sorting.h
	`sorting.h`
	Console Screenshot:

Observations	Selection sort is an algorithm that repeatedly scans the unsorted section of the list for the minimum element and inserts it at its right position. From the code provided, the routine smallest() determines the position of the smallest number, and selectionSort() performs the swap. With time, the sorted section becomes larger until all elements of the list become sorted.

Table 7-3. Selection Sort Algorithm

Code + Console Screenshot	**Code:** **main.cpp**

```
46      // Displaying Insertion Sort (while-loop)
47      std::cout << "\nINSERTION SORT (while-loop)";
48      std::cout << "\n\nUnsorted Array:\n";
49      displayArray(arr, SIZE);
50      insertionSort1(arr, SIZE);
51      std::cout << "\n\nSorted Array Using Insertion Sort (while-loop):\n";
52      displayArray(arr, SIZE);
53
54      std::cout << "\n------------------------------------------------------------\n";
55
56      // Displaying Insertion Sort (for-loop)
57      std::cout << "\nINSERTION SORT (for-loop)";
58      std::cout << "\n\nUnsorted Array:\n";
59      displayArray(arr, SIZE);
60      insertionSort2(arr, SIZE);
61      std::cout << "\n\nSorted Array Using Insertion Sort (for-loop):\n";
62      displayArray(arr, SIZE);
63
```

sorting.h

```
56  // Function to perform Insertion Sort (while-loop)
57  template <typename T>
58  void insertionSort1(T arr[], int maxSize){
59      int k = 1, j, temp;
60
61      while (k < maxSize){
62          temp = arr[k];
63          j = k - 1;
64
65          while(temp <= arr[j]){
66              arr[j+1] = arr[j];
67              j--;
68          }
69          arr[j+1] = temp;
70          k++;
71      }
72  }
```

Console Screenshot:

```
INSERTION SORT (while-loop)

Unsorted Array:
35 32 40 18 81 41 83 14 20 85 22 24 96 91 77 15 79 51 21 50 11 49 22 69 97 20 69 24 64 64 7 93 96 72
 18 76 65 29 52 50 51 53 50 10 87 15 23 88 45 11 17 83 47 80 91 72 22 89 75 41 92 73 14 2 90 14 78 1
9 31 31 47 81 32 28 46 34 13 10 85 91 8 36 75 80 97 42 99 83 54 44 2 80 3 85 66 41 40 48 76 8

Sorted Array Using Insertion Sort (while-loop):
2 3 7 8 8 10 10 11 11 13 14 14 14 15 15 17 18 18 19 20 20 21 22 22 22 23 24 24 28 29 31 31 32 32 34
32766 35 36 40 40 41 41 41 42 44 45 46 47 47 48 49 50 50 50 51 51 52 53 54 64 64 65 66 69 69 72 72 7
3 75 75 76 76 77 78 79 80 80 80 81 81 83 83 83 85 85 85 87 88 89 90 91 91 91 92 93 96 96 97 97 99
------------------------------------------------------------
```

Observations	The while loop in function insertionSort1() starts with k set to 1 and proceeds with the array elements individually. In the meantime, while temp contains the current element being examined, j compares those coming before it. Whenever temp is found to be smaller than the preceding element, the preceding element is shifted one step to the right. This goes on until temp is sorted into its rightful place.

Table 7-4. Insertion Sort Algorithm (while loop)

Code + Console Screenshot	**Code:**
	Main.cpp
	`// Displaying Insertion Sort (for-loop)`
	`std::cout << "\nINSERTION SORT (for-loop)";`
	`std::cout << "\n\nUnsorted Array:\n";`
	`displayArray(arr, SIZE);`
	`insertionSort2(arr, SIZE);`
	`std::cout << "\n\nSorted Array Using Insertion Sort (for-loop):\n";`
	`displayArray(arr, SIZE);`
	
	sorting.h
	`// Function to perform Insertion Sort (for-loop)`
	`template <typename T>`
	`void insertionSort2(T arr[], int maxSize){`
	`for(int k = 1; k < maxSize; k++){`
	`T temp = arr[k];`
	`int j = k - 1;`
	`for(; j >= 0 && temp <= arr[j]; j--){`
	`arr[j+1] = arr[j];`
	`}`
	`arr[j+1] = temp;`
	`}`
	`}`
	
	Console Screenshot:
	`INSERTION SORT (for-loop)`
	
	`Unsorted Array:`
	`48 63 67 38 38 46 18 84 0 10 69 40 92 92 39 51 31 27 0 7 74 56 53 49 4 45 75 34 96 55 3 26 12 39 61`
	`52 37 59 61 70 18 78 1 91 31 90 9 98 91 65 3 86 30 16 14 38 42 14 17 36 56 77 4 69 86 64 19 27 32 33`
	`62 93 59 27 47 21 47 14 62 71 84 74 73 42 12 34 56 21 48 38 5 76 26 90 39 79 48 88 4 62`
	
	`Sorted Array Using Insertion Sort (for-loop):`
	`0 0 1 3 3 4 4 4 5 7 9 10 12 12 14 14 14 16 17 18 18 19 21 21 26 26 27 27 27 30 31 31 32 33 34 34 36`
	`37 38 38 38 38 39 39 39 40 42 42 45 46 47 47 48 48 48 49 51 52 53 55 56 56 56 59 59 61 61 62 62 62 6`
	`3 64 65 67 69 69 70 71 73 74 74 75 76 77 78 79 84 84 86 86 88 90 90 91 91 92 92 93 96 98`

Observations	In function insertionSort2(), the outer for loop starts from index k = 1 and runs through all the elements in the array. The element that is being compared is stored in temp and for loop on the inside compares that element with previous elements. All elements larger than temp are shifted one place to the right.

Table 7-4. Insertion Sort Algorithm (for loop)

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve given data sorting problems using appropriate basic sorting algorithms

Candidate 1	**Bo Dalton Capistrano**
Candidate 2	**Cornelius Raymon Agustín**
Candidate 3	**Deja Jayla Bañaga**
Candidate 4	**Lalla Brielle Yabut**

COMPUTER ENGINEERING

QUEZON CITY

Candidate 5	Franklin Relano Castro

List of Candidates

Problem: Generate an array A[0…100] of unsorted elements, wherein the values in the array are indicative of a vote to a candidate. This means that the values in your array must only range from 1 to 5. Using sorting and searching techniques, develop an algorithm that will count the votes and indicate the winning candidate.

NOTE: The sorting techniques you have the option of using in this activity can be either bubble, selection, or insertion sort. Justify why you chose to use this sorting algorithm.

Deliverables:

- Pseudocode of Algorithm
- Screenshot of Algorithm Code
- Output Testing

PSEUDOCODE:

START

Generate an array that contains 100 votes randomly generated between 1 to 5

Apply Insertion Sort algorithm to sort these votes

Count the votes for each candidate

Compare these counted votes

Find out the candidate having maximum number of votes

Display the winner

STOP

JUSTIFICATION OF SORTING ALGORITHM

I opted for Insertion Sort rather than Bubble Sort and Selection Sort since the values in this activity range from 1 to 5. Due to there being only five values that votes can take, the Insertion Sort will be able to arrange the recurring values in an easy and clear manner. The algorithm also does not need any additional function to obtain the minimum value, which is needed in Selection Sort, and it simply arranges values in the correct positions rather than swapping adjacent values, which happens in Bubble Sort.

RUN 1

Output Console Showing Sorted Array

```
SORTED VOTES:
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2
```

```
VOTE COUNT:
Bo Dalton Capistrano: 23 votes
Cornelius Raymon Agustín: 22 votes
Deja Jayla Bañaga: 18 votes
Lalla Brielle Yabut: 16 votes
Franklin Relano Castro: 21 votes
```

Count Result of Algorithm

```
THE WINNER: Bo Dalton Capistrano with 23 votes.
```

RUN 3

Output Console Showing Sorted Array

```
SORTED VOTES:
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3
 3 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5
5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5
```

Manual Count

```
VOTE COUNT:
Bo Dalton Capistrano: 21 votes
Cornelius Raymon Agustín: 16 votes
Deja Jayla Bañaga: 17 votes
Lalla Brielle Yabut: 25 votes
Franklin Relano Castro: 21 votes
```

Count Result of Algorithm

```
THE WINNER: Lalla Brielle Yabut with 25 votes.
```

QUESTION:

Yes, the developed algorithm for voting proved to be efficient since it was able to sort out the 100 generated votes and compute the number of votes received by each candidate. The sorting function sorted the votes between 1 and 5, and the count loop checked the candidate numbers and tallied their frequencies. The algorithm compared the tallies and found the candidate with the maximum votes as the winner.

CONCLUSION:

From this activity, I have learned the basics of implementing Bubble Sort, Selection Sort, and Insertion Sort in C++, unlike the other activities this was quite easy to digest during the lecture. I have seen how loops, comparisons, swapping and shifting of elements are used in sorting. Furthermore, I have learned that Insertion Sort can be written with a for-loop or with a while-loop. This was a really good review for the basics and in the process I managed to patch some knowledge gaps along the way while coding.

Through the procedure, I have been able to learn about sorting through the implementation of sorting methods after the creation of an array of randomly generated values. The supplemental activity enabled me to see the application of sorting in real-life situations through the organization and counting of votes for the respective candidates using the insertion algorithm.I believe that I have done reasonably well in this activity because I have been able to learn how to implement the basic sorting methods. However, there is a need for me to improve my ability to implement loops and sort elements without having to rely on the provided reference code from the lecture. Nonetheless, this activity was quite chill and I've learned a lot from the lecture and doing the supplementary.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT, Google Gemini and QuillBot**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Used for concepts and misconceptions, most of the code was referenced/reused from the procedure.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.